Comparateurs en Java

La comparaison de deux instances en Java considère un aspect essentiel de l'OOP : on travaille plus avec des variables atomiques (une seule valeur), sinon que avec des instances de classe (objets) qui possèdent plusieurs attributs qu'on peut utiliser pour la comparaison.

Quel que soit le mécanisme de tri, le principe sous-jacent est toujours le même : une comparaison doit établir une relation d'ordre entre deux instances de la même classe. En Java (comme dans des autres langages modernes), les méthodes de comparaison retournent des valeurs entiers tels que :

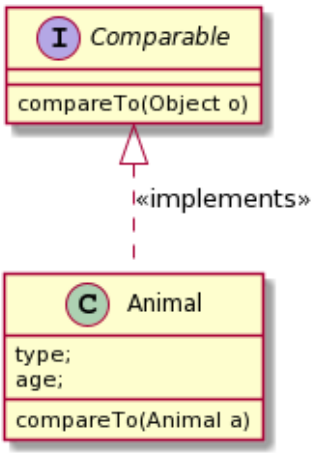
- une valeur entière négative indique qu'un objet A *précède à* un objet B.
- une valeur entière positive indique qu'un objet A *suit à* un objet B.
- la valeur zéro indique que les deux objets sont égaux.

Comparable (*java.lang.Comparable*)

Cette interface permet d'implémenter la méthode `int compareTo(Object o)` directement dans la classe de base. Un exemple bien connu (déjà implémenté en Java) est dans la classe String, où la méthode `compareTo()` est utilisé pour comparer deux chaînes de caractères.

Ce mécanisme a l'avantage d'être rapide à implémenter ; par contre on ne peut pas définir plus d'une méthode `compareTo()` par classe.

Exemple : Considérons une classe `Animal` avec les attributs `type` et `age`. Une implémentation possible de l'interface `Comparable` est :

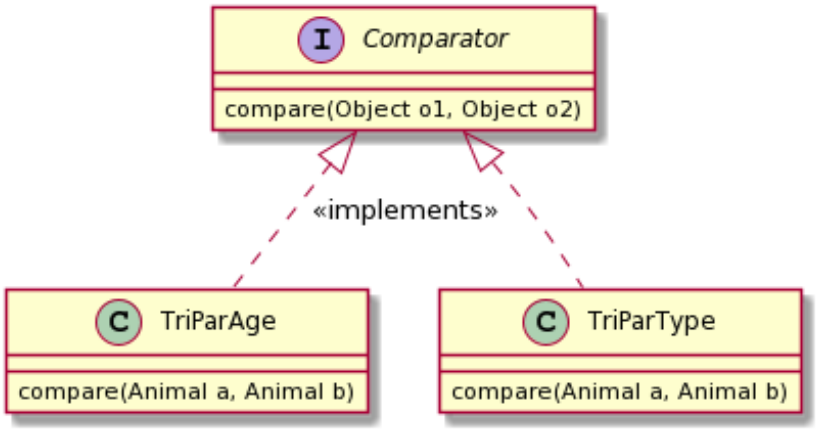



```java
import java.lang.Comparable;

class Animal implements Comparable<Animal>
{
    public String type;
    public int age;
    ...
    public int compareTo(Animal a)
    {
        return (this.age - a.age);
    }
}
```

Comparator (*java.util.Comparator*)

Une deuxième option, plus générique que la première, c'est l'interface `Comparator<>`, qui permet d'implémenter différentes classes de trieurs (ou comparateurs) sur la classe de base.



Classe **TriParAge** :

```
import java.util.Comparator;

class TriParAge implements Comparator<Animal>
{
    public TriParAge(){}
    public int compare(Animal a, Animal b)
    {
        return (a.age - b.age);
    }
}
```

Classe **TriParType** :

```
import java.util.Comparator;

class TriParType implements Comparator<Animal>
{
    public TriParType(){}
    public int compare(Animal a, Animal b)
    {
        return (a.type.compareTo(b.type));
    }
}
```

Collections.sort()

Une **collection** est une manière générique de faire référence aux différentes structures de données qui stockent des ensembles d'objets, par exemple : `List`, `ArrayList` (sous-classe de List), `HashMap`, etc.

Parmi les méthodes associées à cette classe, on trouve `Collections.sort()`, une méthode qui implémente le tri d'objets en ordre ascendant. Les deux versions de cette méthode sont :

- `List<T> sort(List<T> list)`
- `List<T> sort(List<T> list, Comparator<> c)`

Exemple d'utilisation : Considérez les classes déjà definis s

```
public static void main(String[] args) throws Exception {
    ArrayList<Animal> liste = new ArrayList<Animal>();
    liste.add(new Animal("poulet", 1));
    liste.add(new Animal("crocodile", 20));
    liste.add(new Animal("ours", 30));
    liste.add(new Animal("panda", 4));
    // Animal[poulet, 1] Animal[crocodile, 20] Animal[ours, 30] Animal[
       panda, 4]
```

```
        Collections.sort(liste, new TriParAge());
        // Animal[poulet, 1] Animal[panda, 4] Animal[crocodile, 20] Animal[
           ours, 30]

        Collections.sort(liste, new TriParType());
        // Animal[crocodile, 20] Animal[ours, 30] Animal[panda, 4] Animal[
           poulet, 1]
    }
```

Références

[1] Comparable (Java Platform SE 8),
https://docs.oracle.com/javase/8/docs/api/java/lang/Comparable.html

[2] Comparator (Java Platform SE 8),
https://docs.oracle.com/javase/8/docs/api/java/util/Comparator.html

[3] J.M. Doudoux - 12.7 Le tri des collections,
https://www.jmdoudoux.fr/java/dej/chap-collections.htm#collections-7

[4] Comparateur et Comparable en Java,
https://www.codeflow.site/fr/article/java-comparator-comparable